

Grapheon RL: A Graph Neural Network and Reinforcement Learning Framework for Constraint and Data-Aware Workflow Mapping and Scheduling in Heterogeneous HPC Systems

^{1st} Aasish Kumar Sharma

Faculty of Mathematics and Computer Science

Georg-August-Universität Göttingen

Göttingen, Germany

aasish-kumar.sharma@gwdg.de,

orcid: 0000-0002-7514-2340

^{2nd} Julian Kunkel

Faculty of Mathematics and Computer Science

Georg-August-Universität Göttingen

Göttingen, Germany

julian.kunkel@gwdg.de,

orcid: 0000-0002-6915-1179

Abstract—Efficient workflow mapping and scheduling in heterogeneous HPC-Compute Continuum (HPC-CC) systems is critical for multi-objective optimization like optimizing resource utilization and minimizing makespan or energy efficiency. Existing approaches face fundamental trade-offs: Mixed-Integer Linear Programming (MILP) provides optimal solutions but becomes computationally intractable for large workflows exceeding (50x50) nodes by tasks, while heuristic methods sacrifice optimality for speed and struggle with complex constraint modeling. We present *GrapheonRL*, a novel Graph Neural Network (GNN) and Reinforcement Learning (RL)-based framework that can be embedded in Snakemake to model workflows as dependency-aware graphs, enabling RL agents to dynamically learn constraint-aware scheduling policies without mathematical reformulation. We evaluated *GrapheonRL* against MILP and heuristic baselines (HEFT, OLB) on Standard Task Graph Set workflows (11–90 tasks) and extended to synthetic workflows of up to (10,000x10,000) nodes by tasks. *GrapheonRL* matches MILP optimality while offering significantly improved scalability, achieving 76% faster inference with linear memory growth (0.87 MB per 1000 tasks). On complex workflows, *GrapheonRL* maintains optimal makespan (569) whereas heuristics degrade substantially (HEFT: 829, OLB: 1160), demonstrating that learning-based scheduling effectively bridges the optimality-scalability gap for dynamic HPC-CC environments.

Index Terms—Heterogeneous High Performance Computing, Graph Neural Networks, Reinforcement Learning, Mixed Integer Linear Programming, Heuristics, Workflow Scheduling, Multi-Objective Optimization, Snakemake

I. INTRODUCTION

a) Background: Modern HPC systems span a “compute continuum” (HPC-CC) encompassing edge devices, cloud platforms, and supercomputers [1]. These systems leverage massive parallelism to address computationally intensive problems, evolving beyond traditional von Neumann architectures [2]. The diversity of nodes—ranging from CPUs to GPUs and accelerators—enables flexible resource allocation but introduces challenges in managing heterogeneity effectively. Optimizing resource utilization, minimizing data transfer over-

head, and scheduling tasks efficiently across these systems are critical objectives [3], [4].

b) Difficulties of Existing Tools: Despite the capabilities of existing tools, such as SLURM, Kubernetes, and Snakemake, significant limitations remain: *SLURM*: Relies heavily on static scheduling policies and is less adaptive for data transfer and constraints as per the host objective(s). *Kubernetes*: Lacks fine-grained control for HPC workloads and is complex to integrate with traditional HPC environments. *Snakemake*: Focuses on workflow management but ask users for manual resource mapping and optimization based on host resources.

c) The Optimality–Scalability Trade-off of Techniques: Mixed-Integer Linear Programming (MILP) yields provably optimal schedules but scales as $\mathcal{O}(2^n)$, becoming infeasible for larger workflows (~ 100 tasks [5]). Heuristic methods such as HEFT [6], [7] and OLB offer $\mathcal{O}(n \log n)$ runtime yet can incur 10–40% longer makespan compared to optimal solutions [8] like MILP.

d) Motivation and Challenges: To address these limitations, this research aims to: a) provide an advanced optimization models (system & workload) that account for heterogeneous resources, task dependencies, and communication (Data Transfer Time (DTT)) costs. b) apply an adaptable mapping and scheduling model where constraints and objectives can be adjusted quickly [9]. c) propose an optimized method that provides optimum solution like MILP but in comparative time like Heuristics for high scale data.

e) GrapheonRL: Bridging the Optimality-Scalability Gap: We introduce *GrapheonRL*, a Graph Neural Network and Reinforcement Learning framework that achieves near-optimal scheduling performance while scaling to large workflows. GNNs naturally model workflow dependencies and resource relationships through message-passing [10], while RL learns adaptive scheduling policies without requiring mathematical reformulation when constraints evolve [11].

GrapheonRL delivers: (1) *scalable optimization* matching

MILP performance with 76% faster inference and linear memory scaling, (2) *seamless integration* training during Snake-make dry-run and inferencing during execution, (3) *flexible constraints* encoding HPC requirements through reward functions, and (4) *polynomial-time complexity* enabling dynamic scheduling for large workflows.

This extends our compute continuum optimization work [9] by embedding machine learning into practical workflow management systems.

The remainder of this paper is organized as follows. Section II surveys existing scheduling approaches just after the background. Section III describes the GrapheonRL framework. Section IV details our evaluation methodology. Section V presents results and analysis. Finally, Section VI concludes and outlines future directions.

II. RELATED WORK

HPC workflow scheduling has been extensively studied, analyzing systematic reviews [12], [13] confirm that scheduling problems remain NP-hard due to combinatorial complexity and diverse system constraints for different existing categories of approaches. They are categorized into four main paradigms: a) Mathematical, Heuristics (H), Meta-Heuristics (MH), Artificial Intelligence (AI)/ Machine learning (ML), and Quantum Computing Optimization (QC) approaches. We keep QC for future work.

1) *Mathematical Optimization (ILP, MILP)*: Mathematical Linear Programming models, such as *Integer* and *Mixed-Integer* Linear Programming (ILP, MILP) provides globally optimal solutions through constrained optimization formulations for defined objectives [14], [15]. While mathematically rigorous, MILP's $O(2^n)$ complexity limits scalability—workflows exceeding n tasks require prohibitive solving times for even 3 nodes [16], [17]. Linear programming relaxations (constraints) reduce complexity but sacrifice optimality through fractional solutions requiring additional rounding procedures [5].

A. Heuristic and Metaheuristic Methods

Classical heuristics provide rapid solutions [7] through problem-specific rules. *Heterogeneous Earliest Finish Time (HEFT)* ranks tasks by upward rank and assigns to processors minimizing finish time, while *Opportunistic Load Balancing (OLB)* assigns tasks to least-loaded processors [8]. These methods offer computational efficiency but often produce suboptimal schedules due to greedy decisions.

Advanced metaheuristics include Genetic Algorithms [6], [18], Particle Swarm Optimization, and nature-inspired approaches. Recent analysis [13] reveals prevailing reliance on such evolutionary and swarm intelligence strategies in contemporary HPC research. However, these approaches lack optimality guarantees and exhibit high parameter sensitivity.

B. Machine Learning Approaches

Reinforcement Learning has shown promise for adaptive scheduling. Deep Q-Networks [19] and policy gradient meth-

ods [20]–[23] demonstrate improved performance over traditional algorithms, but requires extensive training and careful reward engineering.

Graph Neural Networks naturally model workflow dependencies and resource topologies [24]. Recent applications include task dependency modeling [25], resource topology representation [26], and performance prediction based on historical data.

GNN-RL Integration represents an emerging frontier. Studies by Zeng et al. [27], Li et al. [28], and Pan et al. [29] demonstrate potential for scientific workflow optimization and heterogeneous cluster management, though focusing on specific aspects rather than comprehensive constraint modeling.

1) *Emerging Trends in GNN-RL for HPC*: GNN-RL models are gaining traction for optimizing HPC workloads, enabling intelligent task placement and resource allocation in dynamic environments [22], [23], [30]–[33].

C. Research Gaps and Positioning

Current approaches exhibit critical limitations: (1) no solution bridges MILP optimality with heuristic scalability, (2) constraint evolution requires complete mathematical reformulation, (3) limited real-time performance capabilities, and (4) poor integration with production systems. Recent reviews [13] advocate for hybrid optimization combining heuristics, machine learning, and emerging techniques.

GrapheonRL addresses these gaps through: MILP-level optimality scaling to thousands of tasks, flexible reward-based constraint modeling, real-time inference capabilities, and seamless Snakemake integration. This positions GrapheonRL as a significant advancement contributing to the emerging hybrid optimization paradigm while providing practical solutions to long-standing HPC scheduling challenges.

III. METHODOLOGY

This section presents our system and workload models and their experimental validation framework. We combine controlled trials with empirical data to tune model parameters, verify theoretical formulations, and integrate optimization protocols for accurate workload-completion forecasts and resource allocation in HPC-CC environments. Detailed definitions for these variables and models are provided in [9].

1) *Snakemake Scheduling Techniques*: The Snakemake scheduler initially employs an ILP method and, if no feasible solution is found within 10 seconds cut off threshold, reverts back to a greedy heuristic [3] as illustrated in Algorithm 1.

However, Snakemake is only aware of local host resources, it may not adequately reflect remote host-specific priorities, necessitating manual modifications to incorporate updated constraints and objectives. Therefore, we extend its scheduler class adding features to configure system characteristics and scheduler settings as per our objectives and constraints.

A. GNN-RL for Workload Mapping and Scheduling

We model HPC scheduling as a graph where tasks and resources are vertices and edges represent dependencies and

Algorithm 1 Snakemake ILP-Greedy Scheduler

Require: Workflow DAG, Resource Limits, Job Request**Ensure:** Job Schedule, Updated Resource Pool

```
1: Initialize global resources.
2: Create a queue of ready jobs.
3: while there are ready jobs do
4:   Select a ready job from the queue.
5:   if sufficient resources are available then
6:     Allocate resources to the job.
7:     Submit the job for execution.
8:   else
9:     Return the job to the waiting queue.
10:  end if
11: end while
12:
13: return Job Schedule, Updated Resource Pool
```

data transfers. A GNN embeds each vertex with its attributes (cores, memory, features, duration), yielding a compact representation of workflow and system topology. These embeddings drive a PPO-based RL policy [34] that assigns tasks to nodes to minimize makespan. Resource, feature, and precedence constraints are enforced through reward shaping—penalties for violations and bonuses for efficient allocations—allowing on-the-fly adaptation without reformulating mathematical expressions (Section III-A2). Unlike MILP’s exponential scaling or heuristic oversimplifications, GNN-RL learns scalable, generalizable policies for large, heterogeneous HPC workloads.

1) *Proposed Method: GrapheonRL – Single Environment with Time-Based Concurrency:* GrapheonRL (Algorithm 2) schedules large HPC workflows in a single, unified environment that enforces time-based concurrency, dependency order, and resource constraints. Optionally, we pre-train the policy on MILP-derived assignments for smaller sub-DAGs.

Algorithm 2 GNN-RL Scheduler with PPO

Require: Environment $Env(T, N)$, policy π_θ , value V_ϕ , PPO params $(\gamma, \lambda, \epsilon, K, E)$

```
1: for episode = 1 to  $E$  do
2:    $s \leftarrow Env.reset()$ ,  $D \leftarrow \emptyset$ 
3:   while not done do
4:      $h \leftarrow GNN(T, deps)$ ,  $\tilde{s} \leftarrow (s, h)$ 
5:      $a \sim \pi_\theta(\cdot | \tilde{s})$ ,  $s', r, done \leftarrow Env.step(a)$ 
6:     store  $(\tilde{s}, a, r, s', V_\phi(\tilde{s}), done)$  in  $D$ 
7:      $s \leftarrow s'$ 
8:   end while
9:   compute advantages  $\hat{A}$  via GAE( $\gamma, \lambda$ )
10:  for  $k = 1$  to  $K$  do
11:    update  $\pi_\theta$  and  $V_\phi$  with PPO clip( $\epsilon$ ) and value losses over  $D$ 
12:  end for
13: end for
```

2) *GNN-RL: Algorithm & Pseudocode:*a) *GNN Embedding:*

$$h_i = GNN_\theta(t_i, Nbrs(t_i)), \quad \forall t_i \in T.$$

These embeddings augment the RL state, capturing both task attributes and dependency structure.

b) *PPO-Based Policy:* We employ Proximal Policy Optimization [34]. Given state-action pairs, we compute advantages $\hat{A}_t$ via GAE and returns G_t using the critic $V_\phi(\tilde{s}_t)$. Defining the probability ratio

$$r(\theta) = \frac{\pi_\theta(a_t | \tilde{s}_t)}{\pi_{\theta_{old}}(a_t | \tilde{s}_t)},$$

we maximize the clipped surrogate

$$\mathcal{L}^{CLIP}(\theta) = \mathbb{E}_t \left[\min(r(\theta) \hat{A}_t, \text{clip}(r(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t) \right],$$

and update the critic by minimizing

$$\mathcal{L}^{\text{value}}(\phi) = \mathbb{E}_t [(G_t - V_\phi(\tilde{s}_t))^2].$$

3) *Handling Large Workloads:* We considered two scaling strategies: (1) *Chunking* the workflow into sub-DAGs for concurrent scheduling—introducing complexity in critical-path synchronization; (2) *Unified graph processing*, loading the entire DAG into memory to preserve global time consistency at modest memory cost. We adopt the unified approach (Algorithm 3) and retain the trained model for subsequent inference.

Algorithm 3 Unified HPC-CC Scheduling

```
1: Initialize:  $A \leftarrow \emptyset$ ,  $M \leftarrow 0$ ,  $step \leftarrow 0$ ,  $state \leftarrow 0^{|T|}$ 
2: while not done do
3:    $(i, j) \leftarrow \pi(state)$  {Pick task  $t_i$  on node  $n_j$ }
4:    $step \leftarrow step + 1$ 
5:    $reward \leftarrow 0$ 
6:   if  $step \geq \text{max\_steps}$  then
7:      $reward \leftarrow reward - 50 + 10 \cdot (|T| - |A|)$ 
8:     break
9:   end if
10:  if  $\text{invalid}(t_i, n_j)$  then
11:    continue
12:  end if
13:  Compute  $st, et$ 
14:  Update  $schedules[n_j]$ 
15:   $A[t_i] \leftarrow (n_j, st, et)$ 
16:   $M \leftarrow \max(M, et)$ 
17:   $state[i] \leftarrow 1$ 
18:   $reward \leftarrow reward + 15 - \text{dur}(t_i) + \text{res\_factor}$ 
19:  if  $|A| = |T|$  then
20:     $reward \leftarrow reward + 30 - M$ 
21:     $done \leftarrow \text{True}$ 
22:  end if
23: end while
24: Return:  $A, M$ 
```

B. Snakemake Integration

GrapheonRL plugs into Snakemake in two transparent phases:

a) *Dry-Run Training:* Invoked via `snakemake --dry-run`, GrapheonRL parses the workflow DAG and system topology (SLURM/Kubernetes) to generate training dataset using the framework of [9], which is used for actual execution. Simulated scheduling environments mirror real executions, and the GNN-RL model is trained and validated against MILP solutions where feasible.

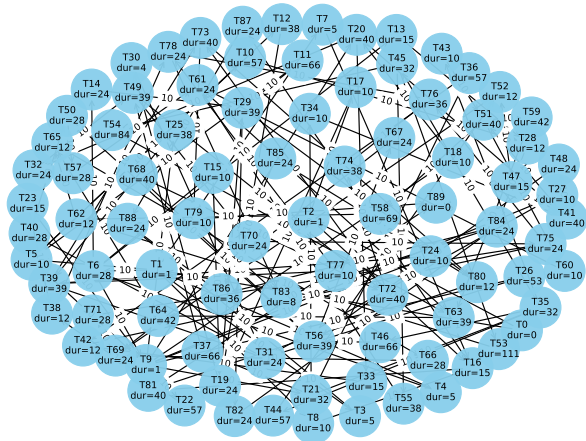


Fig. 1: Workflow 4: Robot control STGS (complex) graph.

b) Execution Inference: At runtime, workflows with > 100 tasks load the pre-trained model in place of Snakemake’s ILP scheduler. Scheduling then runs in task $\mathcal{T}$, node $\mathcal{N}$, $O(|\mathcal{T}| \cdot |\mathcal{N}|)$ time, preserving existing Snakefile rule definitions and executor plugins for seamless, backward-compatible deployment.

C. Complexity and Scalability

Training: One-time cost $O(S \cdot |\mathcal{T}| \cdot |\mathcal{N}|)$ over S dry-run steps. *Inference:* Per-decision cost $O(L \cdot |\mathcal{E}| + |\mathcal{T}| \cdot |\mathcal{N}|)$, enabling real-time scheduling. *Memory:* $O(|\mathcal{T}| + |\mathcal{E}|)$, supporting workflows with 10,000+ tasks. By contrast, MILP scales exponentially. GrapheonRL achieves polynomial complexity while approximating optimal schedules via learned, constraint-aware policies.

IV. EXPERIMENTAL SETUP

A. Test Environment and Datasets

Experiments were conducted on an edge node (8 cores, 8 GB RAM) linked to the HPC-CC; large-scale tests used higher-capacity nodes. All solvers (MILP, GNN-RL, heuristics) are implemented in Python, with PuLP [35] as the MILP backend. We evaluate: a) *Real Workflows:* Four DAGs from the Standard Task Graph Set (STGS) benchmarks [36], [37]. b) *Synthetic Workflows:* Programmatically generated DAGs of varying size and structure (see Table II) for load tests.

B. Test Cases

We compare four STGS-derived workflows: a) *W1:* 10 tasks, zero data-transfer time. b) *W2:* 12 tasks, high per-task DTT. c) *W3:* 11 tasks, default DTT with additional dependencies. d) *W4:* 89 tasks, complex Robot Control graph (Figure 1). Each workflow is scheduled using MILP, GrapheonRL, HEFT, and OLB, and evaluated on makespan, runtime, and resource utilization with three nodes.

C. Evaluation Metrics

1) *Performance:* Performance is assessed based on - *Execution Time and Makespan:* Measured in seconds, *Memory Utilization:* Recorded in MB, and *Task-to-Node Mapping Efficiency:* Evaluated by comparing the sub optimality (constraints violations) against MILP baseline, respectively.

TABLE I: Scheduling performance across workflows (nodes=3).

Workflow	Method	Tasks	Makespan	Time (s)	Mem Δ (MB)
No DTT cost	MILP	11	52	0.02	0.20
	GNN-RL (tr)	11	52	139.41	103.10
	GNN-RL (ts)	11	52	0.01	0.10
	OLB	11	60	0.01	0.01
	HEFT	11	43	0.01	0.01
With DTT cost*	MILP	12	39	0.04	0.20
	GNN-RL (tr)	12	39	143.12	102.90
	GNN-RL (ts)	12	39	0.04	0.10
	OLB	12	42	0.01	0.01
	HEFT	12	35	0.01	0.01
High connections	MILP	11	129	0.08	0.40
	GNN-RL (tr)	11	129	140.81	102.90
	GNN-RL (ts)	11	129	0.06	0.10
	OLB	11	129	0.05	0.04
	HEFT	11	71	0.04	0.03
Robot control	MILP	90	569	0.43	1.50
	GNN-RL (tr)	90	569	176.02	101.40
	GNN-RL (ts)	90	569	1.81	0.10
	OLB	90	1160	0.08	0.06
	HEFT	90	829	0.09	0.05

*Comm. time converted to milliseconds.

GNN-RL emerges as a scalable alternative to MILP for HPC scheduling. MILP delivers optimal makespan but its solve time grows exponentially (0.43 s for 90 tasks). In contrast, GNN-RL’s test phase with training hyperparameters (learning rate: 0.0003, total steps: 80K, max_steps: 1000K) matches MILP’s makespan of 569 in just 1.81 s—76% faster inference—and its train-and-test phase (176.02 s) yields a model that generalizes for rapid scheduling thereafter. HEFT consistently outperforms OLB by 16–30%, especially under heavy dependencies (OLB’s makespan is 1160 vs. MILP’s 569, a 103.8% degradation). For small workflows, MILP (< 0.1 s) remains practical, but for larger DAGs, GNN-RL achieves near-optimal results with only a 0.10 MB memory footprint versus MILP’s 1.5 MB, making it ideal for large-scale HPC environments when considered the training costs. The GNN-RL converged within 790 updates, reducing value loss from ~ 30 K to < 1 K and entropy loss from -5.6 to -2.5 , indicating stable learning.

2) *Makespan (Overall completion time):* As our main object is to minimize makespan by optimally utilizing the available resources. In this regards, the result shown in Figure 2 supports that GrapheonRL is capable to provide optimum makespan based on the given resources similar to MILP in overall, as the result matches with the minimum makespan given by MILP solution.

3) *Scalability:* In regards to the impact of scaling workflow size. For this purpose we utilized a synthetic nodes and workflows generator with a pattern using an arbitrary values

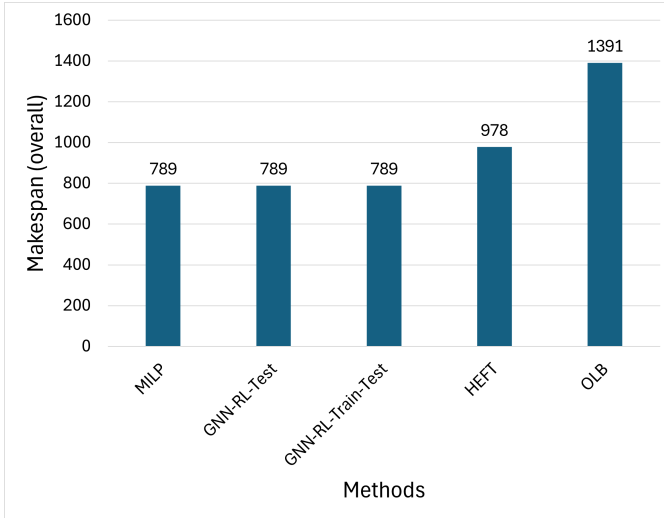


Fig. 2: Overall makespan of methods on different workflows based on Table I.

because our main objective in this test is to verify the workload size that the solver is capable of handling using an edge node.

TABLE II: Scale Test (Runtime) by Method and Problem Size

Method (Behavior)	10×10	100×100	1K×1K	10K×10K
MILP (Optimal)	0.44	—	—	—
GNN-RL-Test (Optimal)	0.65	1.58	27.78	290.40
OLB (Greedy)	0.01	0.09	0.72	75.51
HEFT (Greedy)	0.01	0.11	0.93	107.86

Runtime is in seconds, and $K = 1000$

As shown in Table II, the primitive GNN-RL-Test demonstrates scalability from small to large problem instances while maintaining optimal performance, though with increasing computational cost. At 10×10 tasks, all methods show comparable runtimes under 1 second. However, as scale increases, the trade-offs become apparent: GNN-RL-Test runtime grows from 0.65s to 290.40s at the largest scale, approximately $3.85 \times$ slower than OLB (75.51s) but still achieving optimal solutions. The greedy approaches (OLB and HEFT) remain computationally efficient across all scales, with OLB consistently outperforming HEFT in runtime. MILP becomes intractable beyond the smallest problem size. These results demonstrate that GNN-RL-Test successfully balances solution optimality with reasonable scalability, though at higher computational cost compared to heuristic methods.

V. ANALYSIS AND DISCUSSION

A. Evaluation of Experiments

MILP delivers provably optimal schedules for small workflows but fails to scale beyond a few dozen tasks. Heuristics such as HEFT and OLB run in milliseconds yet can violate complex constraints and yield suboptimal makespan. In contrast, our GNN-RL scheduler achieves a balance of scalability, adaptability, and solution quality by encoding constraints in

the reward function and learning to handle dependencies and resource heterogeneity.

1) *Algorithmic Complexity*: The inference phase of GNN-RL has time complexity $O(MN)$ for M tasks and N nodes, matching the action-state evaluation in each step. Concurrency checks incur an additional $O(MN)$ overhead, and training over S episodes requires $O(SN)$ operations. Although training is more expensive, inference remains efficient and linear in problem size.

2) Performance Comparison:

a) *MILP vs. GNN-RL*: MILP solves workflows up to 12 tasks optimally but becomes infeasible for larger DAGs. GNN-RL maintains within optimal makespan for workflows exceeding 90 tasks, with training times up to 176s and inference times around 1.8s.

b) *Heuristics vs. GNN-RL*: HEFT and OLB schedule in under 0.1s but incur makespan penalties of up to 100% (e.g., 1160 vs. 569 on the Robot Control STG). Furthermore, heuristic makespan remains constant irrespective of task count, whereas GNN-RL adapts to workload scale (e.g., 10,000 tasks).

3) *Scalability and Real-Time Suitability*: GNN-RL scales to 10,000-task workflows while heuristics either lose quality or become static. However, the increased runtime (290.4s vs. 75.5s for OLB) suggests further optimization—such as model compression or parallel inference—is required for strict real-time demands.

4) *Applicability*: *MILP*: Ideal for small, deterministic schedules where optimality is paramount. *Heuristics*: Appropriate for minimal-latency, approximate scheduling under simple constraints. *GNN-RL*: Best suited for large-scale, dynamic HPC workloads needing adaptability to evolving constraints.

VI. CONCLUSION AND FUTURE WORK

We have demonstrated that GNN-RL effectively captures resource concurrency, task dependencies, and scheduling constraints, offering a scalable alternative to MILP and heuristic methods. Although training incurs significant overhead, inference is both accurate and fast—76% quicker than MILP and only $3.85 \times$ slower than OLB—making GNN-RL viable for large, adaptive workflows.

Future work includes: Exploring advanced GNN architectures (e.g., attention-based, hierarchical) to improve representation capacity. Integrating online RL for continuous, real-time scheduling adaptation. Developing distributed or hierarchical GNN-RL schemes to handle extreme-scale workflows ($> 10^5$ tasks).

ACKNOWLEDGMENTS

This work is supported by the European Horizon Project DECICE (Grant No. 101092582) and the National High Performance Computing (NHR) initiative. We thank GWDG for their significant contributions to this research. The implementation source code is available at <https://github.com/AasishKumarSharma/GrapheonRL>.

REFERENCES

- [1] J. Dongarra, "Hpc forecast: Cloudy and uncertain," *Communications of the ACM*, vol. 65, no. 5, pp. 56–66, 2022.
- [2] M. D. Godfrey, "Introduction to "the first draft report on the edvac" by john von neumann," *Annals of the History of Computing*, vol. 15, pp. 11–21, 1993. [Online]. Available: <https://web.archive.org/web/20130314123032/http://qss.stanford.edu/~godfrey/vonNeumann/vnedvac.pdf>
- [3] J. Koster and S. Rahmann, "Snakemake—a scalable bioinformatics workflow engine," *Bioinformatics*, vol. 28, no. 19, pp. 2520–2522, 2012. [Online]. Available: <https://doi.org/10.1093/bioinformatics/bts480>
- [4] A. B. Yoo, M. A. Jette, and M. Grondona, "SLURM: Simple linux utility for resource management," in *Proceedings of the 9th Workshop on Job Scheduling Strategies for Parallel Processing (JSSPP)*. Springer, 2003, pp. 44–60. [Online]. Available: https://doi.org/10.1007/10968987_3
- [5] W.-Y. Ku and J. C. Beck, "Mixed Integer Programming models for job shop scheduling: A computational analysis," vol. 73, pp. 165–173. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0305054816300764>
- [6] C. Blum and A. Roli, "Metaheuristics in combinatorial optimization: Overview and conceptual comparison," *ACM Computing Surveys (CSUR)*, vol. 35, no. 3, pp. 268–308, 2003.
- [7] X. Liao *et al.*, "A heuristic task scheduling approach in cloud computing systems," in *International Conference on Advanced Cloud and Big Data (CBD)*. IEEE, 2014, pp. 40–47.
- [8] R. NoorianTalouki, M. Hosseini Shirvani, and H. Motameni, "A heuristic-based task scheduling algorithm for scientific workflows in heterogeneous cloud computing platforms," *Journal of King Saud University - Computer and Information Sciences*, vol. 34, pp. 4902–4913. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1319157821001269>
- [9] A. K. Sharma, C. Boehme, P. Gelß, R. Yahyapour, and J. Kunkel, "Workflow-driven modeling for the compute continuum: An optimization approach to automated system and workload scheduling," *arXiv preprint arXiv:2505.12184*, 2025.
- [10] J. Zhou, G. Cui, S. Hu, Z. Zhang, C. Yang, Z. Liu, L. Wang, C. Li, and M. Sun, "Graph neural networks: A review of methods and applications," *AI open*, vol. 1, pp. 57–81, 2020.
- [11] R. S. Sutton, A. G. Barto *et al.*, *Reinforcement learning: An introduction*. MIT press Cambridge, 2018, vol. 1, no. 1.
- [12] I. Ahmad, M. G. AlFailakawi, A. AlMutawa, and L. Alsalkan, "Container scheduling techniques: A survey and assessment," *Journal of King Saud University - Computer and Information Sciences*, vol. 34, no. 7, pp. 3934–3947, 2022. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1319157821000562>
- [13] A. K. Sharma and J. Kunkel, "A review of tools and techniques for optimization of workload mapping and scheduling in heterogeneous hpc system," *arXiv preprint arXiv:2505.11244*, 2025.
- [14] D. Bertsimas, *Introduction to Linear Optimization*. Belmont, Mass.: Athena Scientific. [Online]. Available: <http://archive.org/details/introductiontoli0000bert>
- [15] D. P. Bertsekas and J. N. Tsitsiklis, *Parallel and Distributed Computation: Numerical Methods*, 1989th ed. [Online]. Available: <http://athenasc.com/pdcbbook.html>
- [16] T. Achterberg, T. Berthold, T. Koch, and K. Wolter, "Constraint Integer Programming: A New Approach to Integrate CP and MIP," in *Integration of AI and OR Techniques in Constraint Programming for Combinatorial Optimization Problems*. L. Perron and M. A. Trick, Eds. Springer Berlin Heidelberg, vol. 5015, pp. 6–20. [Online]. Available: http://link.springer.com/10.1007/978-3-540-68155-7_4
- [17] R. E. Bixby, "Mixed-integer programming solvers," *Wiley Encyclopedia of Operations Research and Management Science*, vol. 5, pp. 1–13, 2012.
- [18] Y. Xu, K. Li, T. T. Khac, and M. Qiu, "A Multiple Priority Queueing Genetic Algorithm for Task Scheduling on Heterogeneous Computing Systems," in *2012 IEEE 14th International Conference on High Performance Computing and Communication & 2012 IEEE 9th International Conference on Embedded Software and Systems*, pp. 639–646. [Online]. Available: <https://ieeexplore.ieee.org/document/6332230>
- [19] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller, "Playing atari with deep reinforcement learning," in *NIPS Deep Learning Workshop*, 2013.
- [20] Y. Gu, Z. Liu, S. Dai, C. Liu, Y. Wang, S. Wang, G. Theodoropoulos, and L. Cheng, "Deep reinforcement learning for job scheduling and resource management in cloud computing: An algorithm-level review." [Online]. Available: <http://arxiv.org/abs/2501.01007>
- [21] S. Mangalampalli, G. R. Karri, M. V. Ratnamani, S. N. Mohanty, B. A. Jabr, Y. A. Ali, S. Ali, and B. S. Abdullaeva, "Efficient deep reinforcement learning based task scheduler in multi cloud environment," vol. 14, no. 1, p. 21850, publisher: Nature Publishing Group. [Online]. Available: <https://www.nature.com/articles/s41598-024-72774-5>
- [22] G. Zhou, W. Tian, R. Buyya, R. Xue, and L. Song, "Deep reinforcement learning-based methods for resource scheduling in cloud computing: a review and future directions," vol. 57, no. 5, p. 124. [Online]. Available: <https://doi.org/10.1007/s10462-024-10756-9>
- [23] Y. Song, C. Li, L. Tian, and H. Song, "A reinforcement learning based job scheduling algorithm for heterogeneous computing environment," vol. 107, p. 108653. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0045790623000782>
- [24] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, "Neural message passing for quantum chemistry," in *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017, pp. 1263–1272.
- [25] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *International Conference on Learning Representations (ICLR)*, 2017.
- [26] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, "Graph attention networks," in *International Conference on Learning Representations (ICLR)*, 2018.
- [27] M. Zeng, M. Zhang, W. Chen, and J. Li, "A graph neural network approach to multi-job scheduling in high-performance computing," in *Proceedings of the IEEE International Parallel and Distributed Processing Symposium Workshops*. IEEE, 2020, pp. 915–924.
- [28] Q. Li, M. Zhao, and N. Helian, "Learning to schedule scientific workflows with deep graph neural networks and reinforcement learning," in *Proceedings of the IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing*. IEEE, 2022, pp. 535–542.
- [29] L. Pan, S. Wu, and L. Zheng, "Gnn-based scheduling for heterogeneous hpc cluster," *Journal of Parallel and Distributed Computing*, vol. 145, pp. 104–117, 2021.
- [30] K. C. Adimora and H. Sun, "GNN-RL: An Intelligent HPC Resource Scheduler," *ACM*, 2024. [Online]. Available: https://sc24.supercomputing.org/proceedings/poster/poster_files/post137s2-file3.pdf
- [31] S. Yu, A. Mazaheri, and A. Jannesari, "Topology-aware network pruning using multi-stage graph embedding and reinforcement learning," in *Proceedings of the 39th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 162. PMLR, 17–23 Jul 2022, pp. 25 656–25 667. [Online]. Available: <https://proceedings.mlr.press/v162/yyu22e.html>
- [32] S. Yu, P. Nguyen, W. Abebe, W. Qian, A. Anwar, and A. Jannesari, "Spatl: Saliency parameter aggregation and transfer learning for heterogeneous federated learning," in *2022 SC22: International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*. Los Alamitos, CA, USA: IEEE Computer Society, nov 2022, pp. 495–508.
- [33] S. Yu, A. Mazaheri, and A. Jannesari, "Auto graph encoder-decoder for neural network pruning," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. IEEE/CVF International Conference on Computer Vision (ICCV), October 2021, pp. 6362–6372.
- [34] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal Policy Optimization Algorithms. [Online]. Available: <http://arxiv.org/abs/1707.06347>
- [35] Optimization with PuLP — PuLP 2.9.0 documentation. [Online]. Available: <https://coin-or.github.io/pulp/>
- [36] T. Tobita and H. Kasahara, "A standard task graph set for fair evaluation of multiprocessor scheduling algorithms," *Journal of Scheduling*, vol. 5, no. 5, pp. 379–394, 2002.
- [37] Kasahara and Narita, "Practical Multiprocessor Scheduling Algorithms for Efficient Parallel Processing," vol. C-33, no. 11, pp. 1023–1029. [Online]. Available: <https://ieeexplore.ieee.org/document/1676376>